10 - Trees

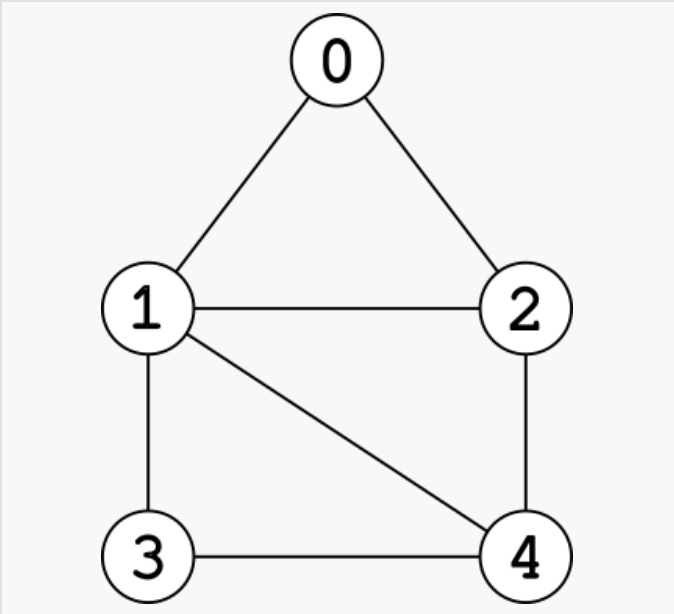
All data structures can be expressed as graphs. For example, we can also define a linked list as a graph because it is composed of vertices (nodes) and edges. We can claim that all linked lists are directed graphs, whether singly linked or double linked, because the links themselves are naturally directed. Two nodes in a linked list are adjacent if there is a reference to one or the other linking them together. Additionally, the linking of nodes constructs a path that connects the head and the tail and this path connects all nodes of the linked-list to the head which is why we can access any element from the head of a linked-list. If a linked list is non-circular and singly-linked, it contains no cycles; however, if it is doubly-linked, there are many possible cycles in the linked list.

Trees are hierarchical data structures and all trees are graphs. With a basic introduction to graph properties, we can develop a general tree definition:

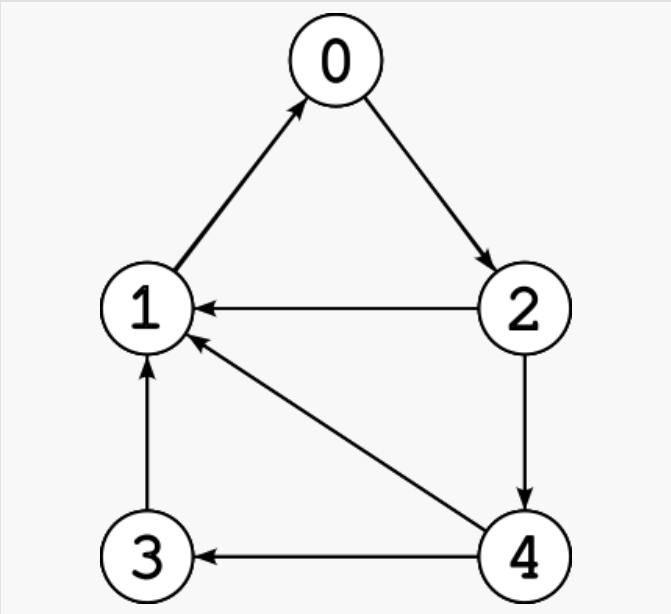
> A Tree is an acyclic, directed graph where all nodes are connected to a single node called the root.

Introductory Graph Properties

We will revisit graphs at the end of the course; however, it is useful to introduce some ideas now.



Basic Graph



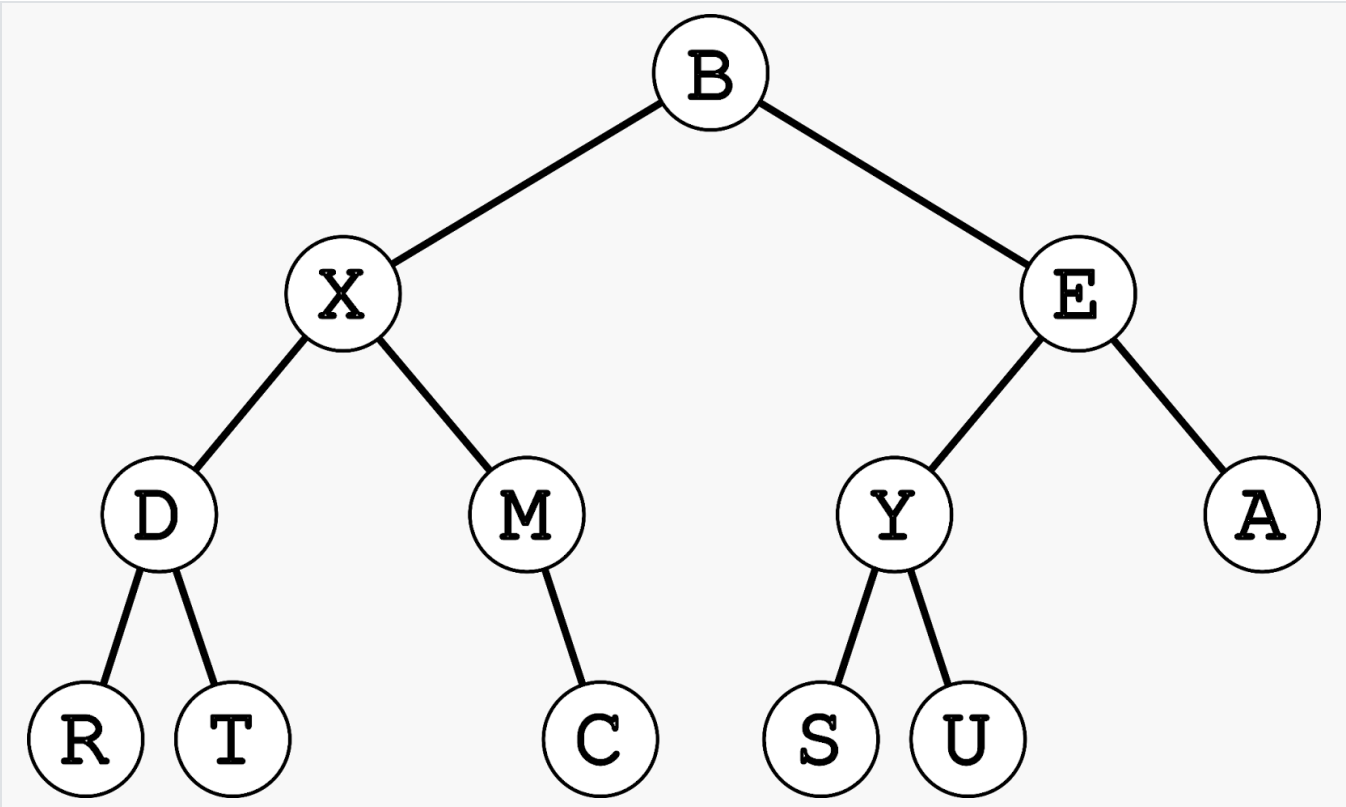
Directed Graph

A **graph** G is defined by $G = (V, E)$ where V is a set of vertices and E is a set of edges. An **edge** is a pair of vertices in the graph where edge e_k is defined as $e_k = (v_1, v_2)$. The graph may be **undirected** allowing for edges to be traversed in either direction, from v_1 to v_2 or from v_2 to v_1 allowing the vertex pair to be unordered.

In a **directed** graph, edges have direction and edges can only be traversed along their direction, so the vertex pair may define the direction through its order, $e_k = (\overrightarrow{v_1, v_2})$, meaning that this edge only defines traversal from v_1 to v_2. In most directed graph diagrams, direction may be explicitly illustrated by arrowheads that are interpreted as the terminating end of an edge but direction may also be implied using other conventions, *e.g.* a tree diagram uses a tiered structure to imply direction.

Two vertices are **adjacent** or neighbors if there is an edge between them. Two nodes are **connected** if there is a set of adjacent vertices, *i.e.* a **path**, such that it is possible to traverse from the first vertex to the second vertex through the graph. For a directed graph, a directed path must follow any direction constraints. A **cycle** exists if there is a path connecting a vertex to itself and a graph with no cycles is called **acyclic**. Generally, the **distance** between vertices v_i and v_j is the number of edges in the path connecting v_i and v_j and the **shortest path** between v_i and v_j is the path with the minimum distance.

Introduction to Trees



Example of a tree

The vertices of a tree are frequently referred to as nodes and vertex and node terminology can be used interchangeably.

The **root** node of a tree is a singular node to which all other nodes in the tree are connected. In the example above, the root of the tree is node B.

A **branch** is an edge connection in a tree. In the example above, there are many branches joining nodes in the tree, but some examples of branches are the edges (B, E), (B, X), (X, M), and (Y, S).

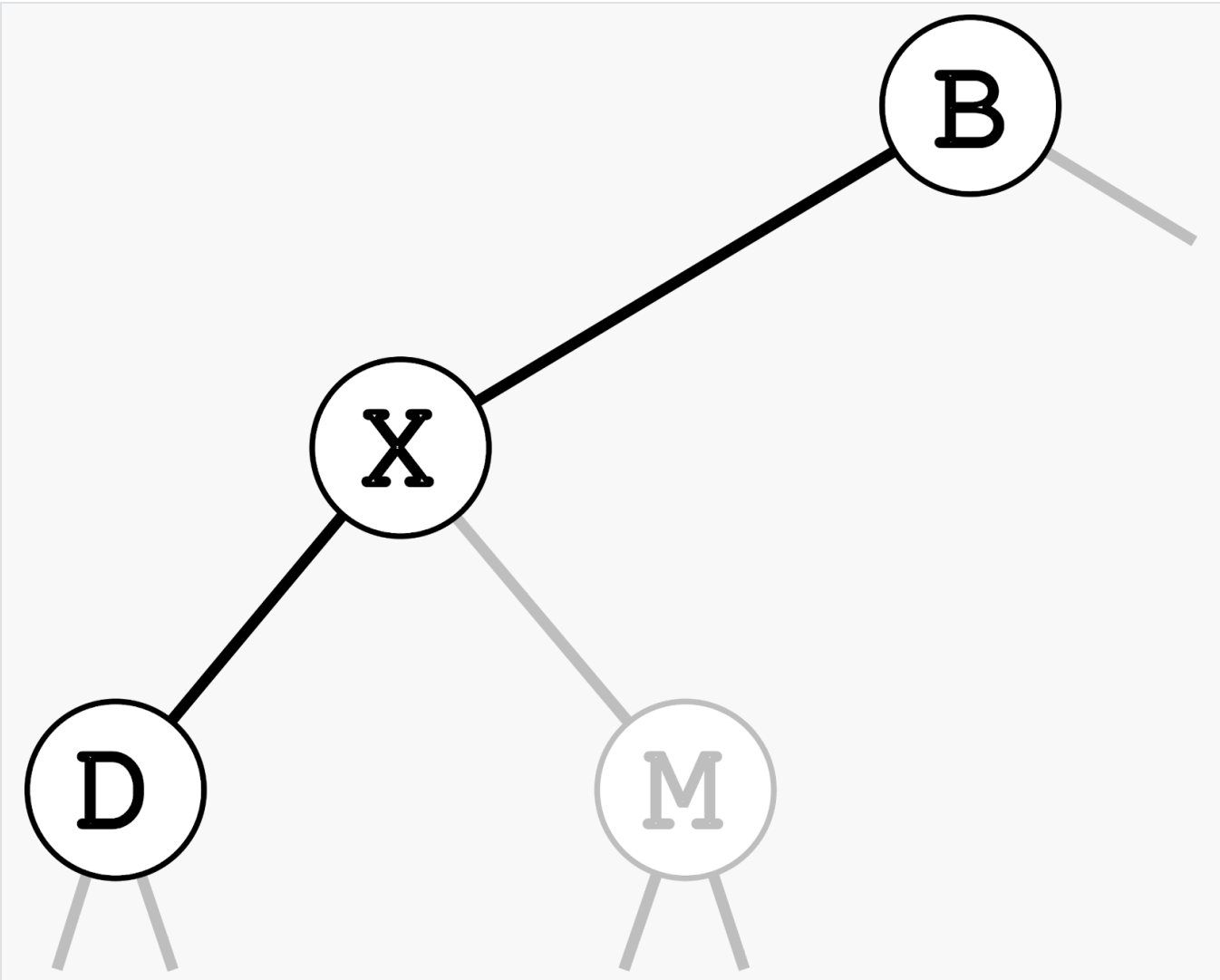
A tree is naturally directed originating from the root. In the edge convention syntax $(\overrightarrow{v_1, v_2})$ for directed graphs, direction is implied by the ordering of the vertices in the pair, so for the edge (B, E), the edge originates at the first vertex specified, B, and terminates at the second node specified, E, therefore, the edge is directed from B to E.

The direction of edges is also implied by the hierarchical vertical ordering of the tree. Since edges are directed from the root down the tree, given the edge $(\overrightarrow{v_1, v_2})$, vertex v_1 is higher in the tree (closer to the root) and vertex v_2 is lower (further from the root) in the tree. Therefore the edge is directed from v_1 to v_2.

A tree is technically a tree of trees. In other words, each node is the root of a subtree containing the nodes that are connected to and lie below that node. In the above example, D is the root of a subtree containing nodes D, R, and T. This should suggest that many algorithms that operate on trees can be decomposed into self-similar subproblems and recursive algorithms are especially useful in trees.

A tree node is either an inner node or an outer node. An **inner node** is connected to at least one node further down the tree and may also be called a **branch node**. In the above example, nodes B, X, M, and Y are inner nodes. An **outer node** is not connected to any nodes further down the tree and is also called a **leaf node** or **terminal node**. In the above example, R, C, and A are outer nodes.

If there is a directed path from node n_1 to node n_2, *i.e.* n_1 is closer to the root and n_2 exists within the subtree where n_1 is the root, then n_1 is an **ancestor** of n_2 and n_2 is a **descendant** of n_1, and two nodes are said to have a **parent-child relationship** if they are neighbors. In other words, X is the parent node of D and the child node of B. We might also say D is the **grandchild** of B and B is the **grandparent** of D.



X is the parent of D because D is both adjacent to X and descends from X. X is also the child of B because X is both adjacent to B and descends from B. D is the grandchild of B because D is the child of X and X is the child of B. Conversely, we can say B is the grandparent of D because B is the parent of X and X is the parent of D.

Metrics

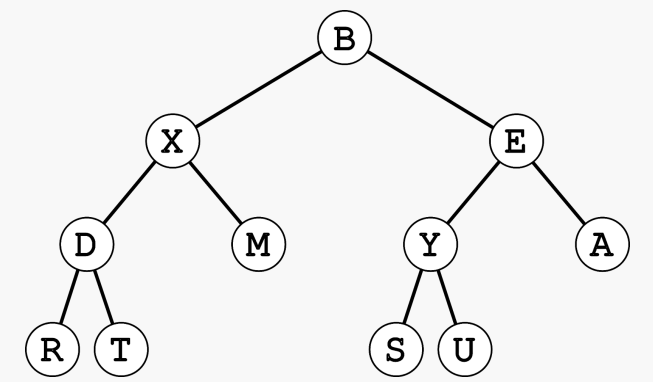
The **degree of a node** is the number of outgoing branches that originate from this node; therefore, an inner node has a degree greater than zero and an outer node has a degree of zero. The **degree of a tree** is the maximum degree of any node in the tree. In the above example, the degree of the tree is 2 because the maximum outgoing degree of any of the nodes in the tree is 2.

As defined for graphs, the **distance** between node n_1 and node n_1 is the number of edges between the two nodes in the path connecting the two nodes. Because trees are acyclic by definition, all trees have only one connection between two nodes, so it is only possible for there to be one path between two nodes. All tree nodes are connected to its root, so It is always possible to reach any node in the tree from the root.

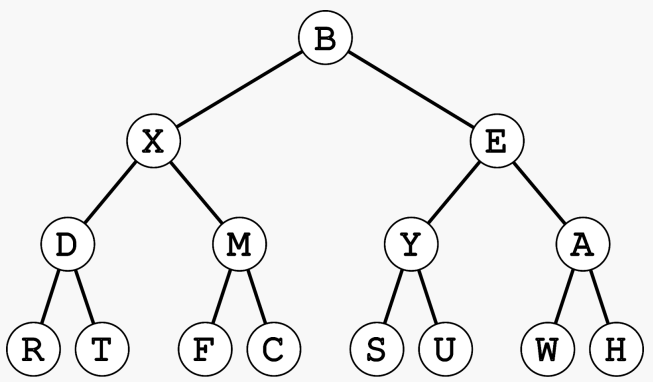
The **depth of a node** or **level of a node** is the distance of a node from the root in terms of the number of edges between the root and that node. The root is the root and there are no edges between the root and itself, so the distance between the root and itself is 0 meaning that the root is at depth/level 0. Any node that is a child of the root is at depth 1 or level 1, any node that is the grandchild of the root is at depth 2, and so forth. The **depth of the tree** or **height of the tree** is the distance of the furthest leaf from the root and represents the longest path from the root to any leaf in the tree.

Binary Tree and Binary Search Tree

A tree with degree of 2 is called a **binary tree**. A binary tree that contains inner nodes with only a degree of 2, *i.e.* it has no inner nodes with a degree of 1, is a **full binary tree**. If all outer nodes in a full tree only exist at the full depth of the tree, it is a **complete binary tree**.



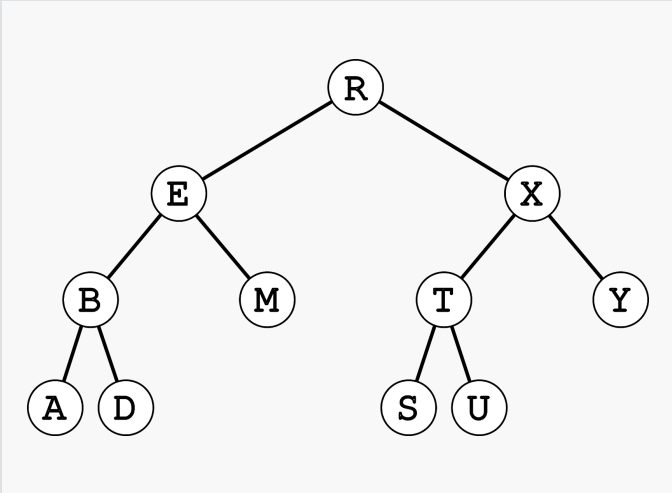
Full binary tree



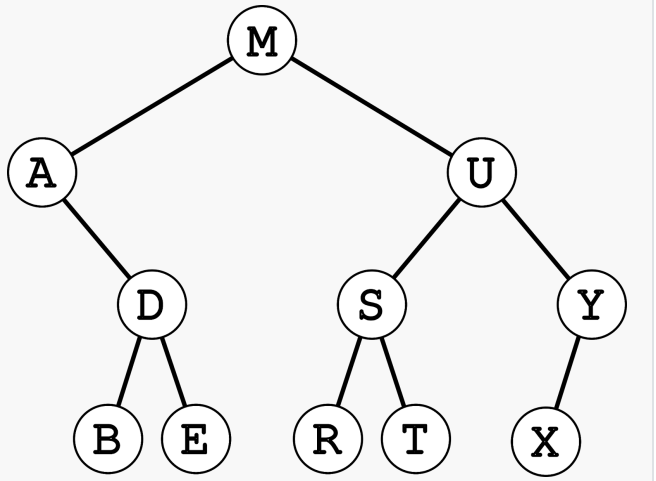
Complete binary tree

A complete binary tree is a tree of **minimum height**, and with balancing, a full binary tree approaches its minimum height. In a binary tree with minimum height, the longest path from root to any leaf is no longer than $\log_2 n$.

An **ordered** binary tree is structured in such a way that search in the tree results in a binary search. If a binary tree is ordered and of minimum height, we call it a **Binary Search Tree (BST)**. Note that not all ordered binary trees are binary search trees because a BST must also have minimum height. Both the full binary tree and complete binary tree examples above are **unordered** so binary search cannot be performed on them and therefore they are not BST's. A binary search tree has the property that in any subtree given the value of its root then all nodes in each subtree with values less than that subtree's root value lie to the left of the root, otherwise, the value must lie to the right of the root node. However, there is no single ordering for a BST and there are many potential orderings for the same data. Both of the following trees are ordered binary trees of minimal height, so both are BST's. The left example 1 is full while the right example 2 is not full.



Ordered binary tree example 1



Ordered binary tree example 2

Search Analysis

There is a close relationship between binary search in an ordered array and search in an ordered binary tree; however, there are sufficient differences that the efficiency analysis for each structure is unique. The most important distinction is that binary search in an ordered array always results in the search of a binary tree of minimum height (we leave it to the reader to examine this further) and the search efficiency follows from this fact. However, the efficiency of search in an ordered binary tree is subject to whether or not the tree's height is minimized which represents a significant variable in efficiency analysis and key factor in tree implementation.

For the sake of analysis, assume a complete, ordered tree meaning that the tree is of minimum height and search us guaranteed to follow a single path. Our worst case analysis therefore seeks to quantify the length of the longest path from root to leaf in a complete, ordered tree. In other words, what is the relationship between the length of the longest path and the number of elements in the complete, ordered binary tree.

For each level, ℓ, of the complete binary tree, the number of elements in a given level, n_ℓ , is equal to the base 2 exponential of the level where the first level containing the root node is level 0. In other words, the relationship between the level and the number of elements in a level for a complete binary tree is:

$$n_\ell = 2^\ell$$

This formula makes sense because the first level, $\ell = 0$, contains only the root making $n_0 = 2^0 = 1$, the second level, $\ell = 1$, contains the two children of the root in a complete binary tree making $n_1 = 2^1 = 2$, and each subsequent level doubles the number of elements from the previous level.

The depth of the tree, d, is equal to the maximum level and the total number of elements in the tree would therefore be the sum of the elements in all levels up to the depth of the tree:

$$n = n_0 + n_1 + \cdots + n_d$$

Therefore the total number elements in the complete binary tree is the summation:

$$n = \sum_{\ell=0}^{d} 2^{\ell}$$

This summation establishes that for a complete tree of depth $d = 0$, the total number of elements in the tree is:

$$n = n_0 = 2^0 = 1$$

And for a complete tree of depth $d = 1$, the total number of elements in the tree is:

$$n = n_0 + n_1 = 2^0 + 2^1 = 1 + 2 = 3$$

And for a complete tree of depth $d = 2$, the total number of elements in the tree is:

$$n = n_0 + n_1 + n_2 = 2^0 + 2^1 + 2^2 = 1 + 2 + 4 = 7$$

Consider the relationship of the above summation and examples with the number of elements in the next level added beyond the current depth of a complete tree, *i.e.* n_{d+1}. When a new level is introduced, the total number of elements in the next level is equal to:

$$n_{d+1} = 2^{d+1}$$

But what is the relationship between the number of elements in the new level with all the existing elements in the tree? From above:

$$n_0 + n_1 + n_2 = 7$$

and

$$n_{2+1} = 2^{2+1} = 8$$

therefore

$$n_0 + n_1 + n_2 = n_{2+1} - 1$$

or expressed in terms of the total number of elements in the tree n:

$$n = n_{d+1} - 1$$

While this text only proves this relationship for a narrow specific example, it holds for all levels in a complete binary tree. For example, $n_0 = 1 = n_{0+1} - 1 = 2 - 1$ and $n_0 + n_1 = 1 + 2 = n_{1+1} - 1 = 4 - 1$. We leave it to you to refer to the proof for the sum of a geometric series for a full proof on this relationship.

The above means that the relationship between the number of nodes n in a complete binary tree and the depth d is:

$$n = 2^{d+1} - 1$$

This formula is the answer to our question about the relationship between the longest path and the number elements; however, it does not answer the question about what is the length of the longest path directly. To determine the length of the longest path, we need to determine what the depth d would be for a complete binary tree in terms of the number of elements in the tree n, so we should probably rearrange the equation first:

$$2^{d+1} + 1 = n$$

Since we are performing Big O analysis, we are estimating and can make some slight simplifications to reduce complexity in the equation:

$$2^d \approx n$$

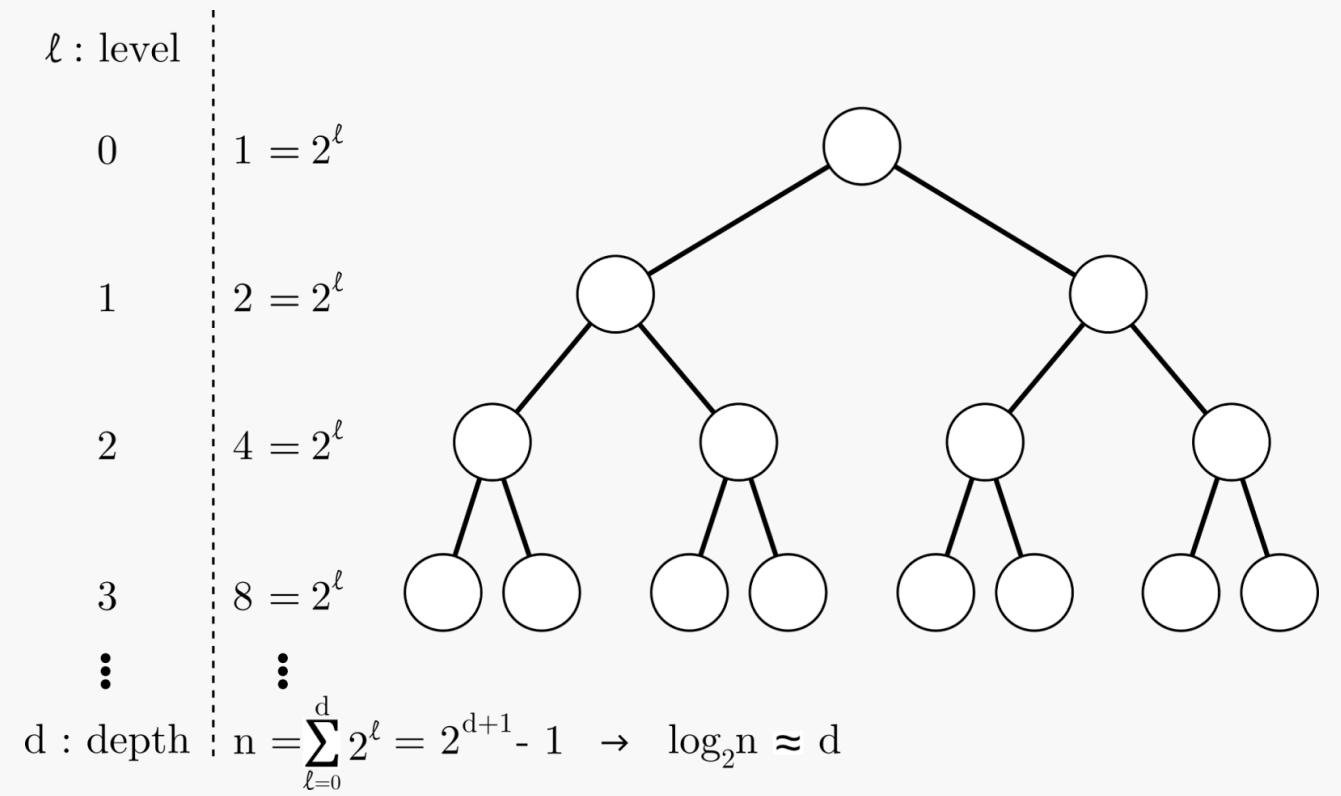
We can then apply the logarithm to both sides to isolate and express in terms of d:

$$\log_2(2^d) \approx \log_2 n$$

Which reduces to:

$$d \approx \log_2 n$$

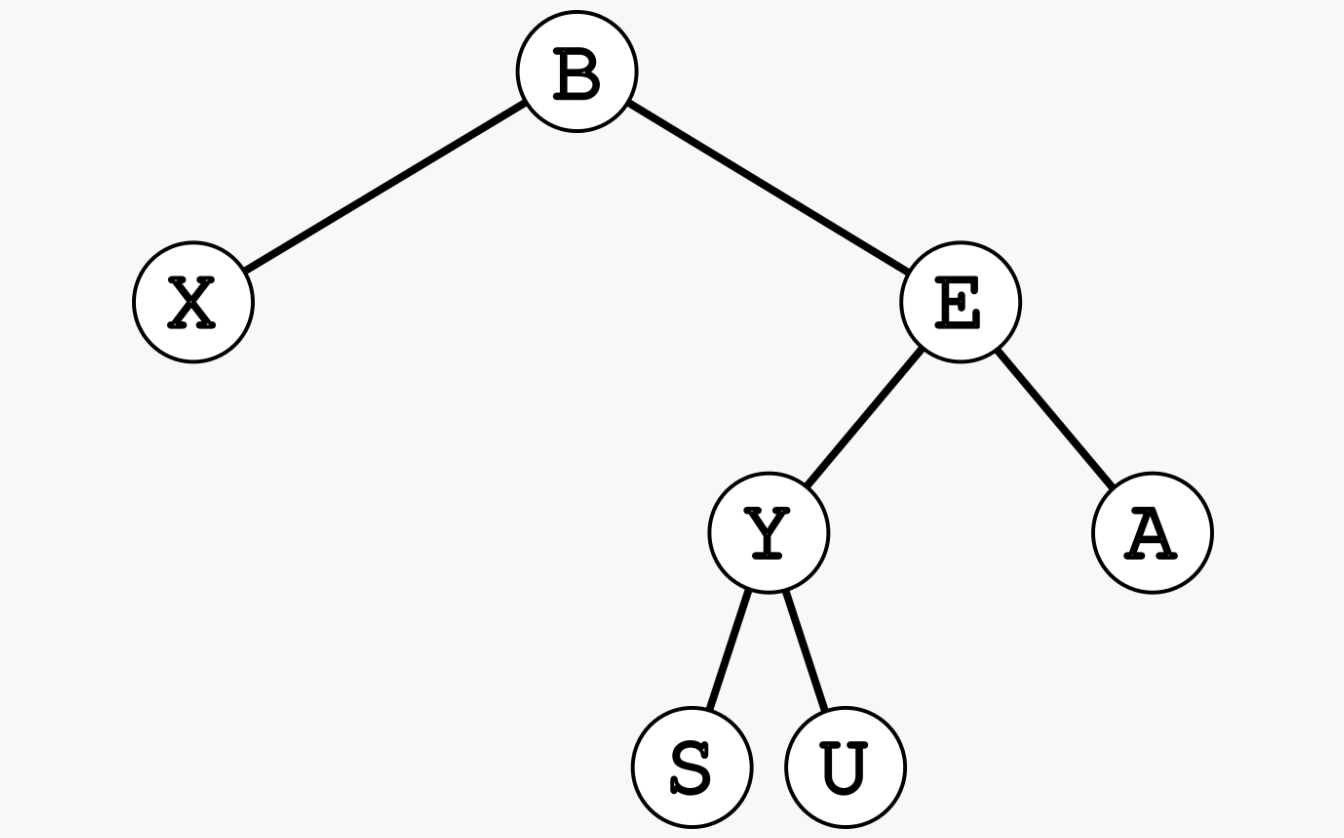
In other words, in a complete binary tree, no node is more than $\log_2 n$ edges from the root where n is the number of elements in the tree. This means that to search an ordered, complete binary tree, *i.e.* a complete BST, it takes no more than $\log_2 n$ comparisons to determine whether or not the element in question exists in the complete BST and search in a complete BST is therefore $O(\log_2 n)$.



For a complete binary tree, the total number of elements n is related to the depth d by $n = 2^{d+1} - 1$, the base 2 exponential, or alternatively via by $\log 2n \approx d$, the base 2 logarithm. This means that the distance of the furthest leaf from the root is $log_2 n$, and the longest path from the root to any node is no more than the depth of the tree, $\log 2n$, as long as the tree is a complete binary tree.

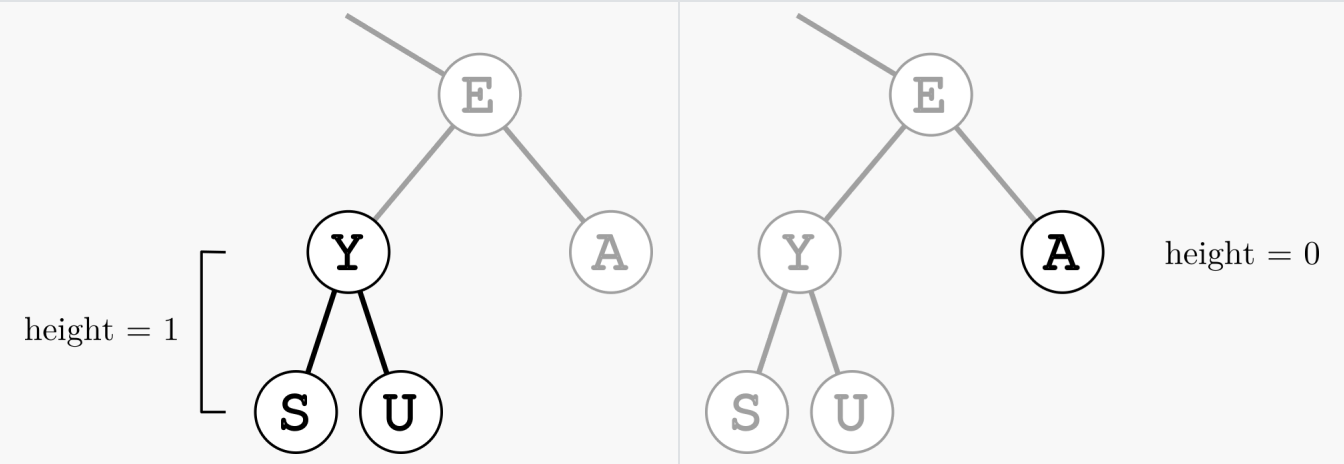
In a complete tree, the relationship between the number of elements and the longest path is $\log_2 n$. Because of the ordering of a BST, a node can only exist on a single path through the tree as dictated by ordering relative to subtree root nodes. This means that there is one and only one search path within a given tree for a specific element, and if that element is not in that path, then the element must not exist within the tree. Therefore, we only need to perform a number of comparisons equal to the length of that path to determine if an element is not in the tree and the length of that path determines the efficiency of the search operation. In other words, if the tree is of minimum height, then the longest path is of length $\log_2 n$ which defines the number of comparisons necessary in a worst case search and therefore the worst-case time complexity for search within a complete BST is $O(\log_2 n)$. This efficiency of search holds because the height of a complete binary tree is minimized by definition, but how does this change for binary search trees in general?

Whether or not a binary search tree performs search with logarithmic time complexity will depend on whether the binary search tree is **balanced**, which is a property specific to algorithms designed to maintain a binary search tree with minimal height.

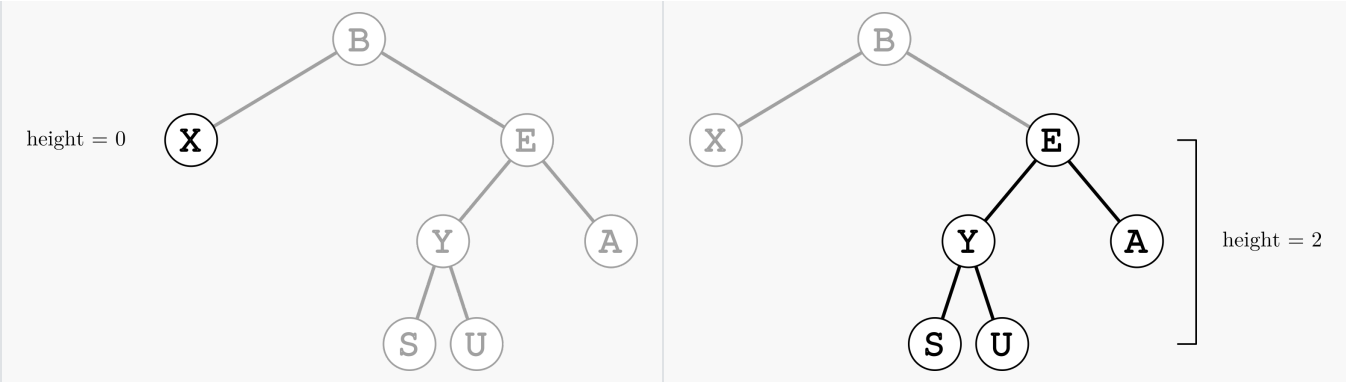


An unbalanced tree

An **unbalanced** tree is generally measured by the height difference between all left and right subtrees for every node in the tree. Note that this is a recursive definition. If this difference between heights in all left and right subtrees is greater than 1 then the tree is unbalanced and needs **rebalancing**.



The left and right subtrees of E have a height difference of 1 so the subtree defined by E as the root is considered balanced



The left and right subtrees of B have a height difference greater than 1 so the subtree defined by B as the root is considered unbalanced

In binary search on an array, the tree that is generated is always ordered (assuming the data is ordered), always has minimum height, and is always balanced, so binary search in an array is always logarithmic time complexity. However, there is no such guarantee that all trees have minimum height unless they are full.

Balance is not a passive property as balance must be regularly maintained, potentially with each insertion into or deletion from a tree, so a key feature of all binary search tree algorithms is to **rebalance** (to rearrange the tree with time complexity less than or equal to the efficiency of search) a tree when necessary, to both maintain its order and minimize its height. If a tree is balanced efficiently, the worst-case search efficiency consistently remains $O(\log_2 n)$ for insert and search in a balanced binary search tree where n is the number of nodes in the tree.

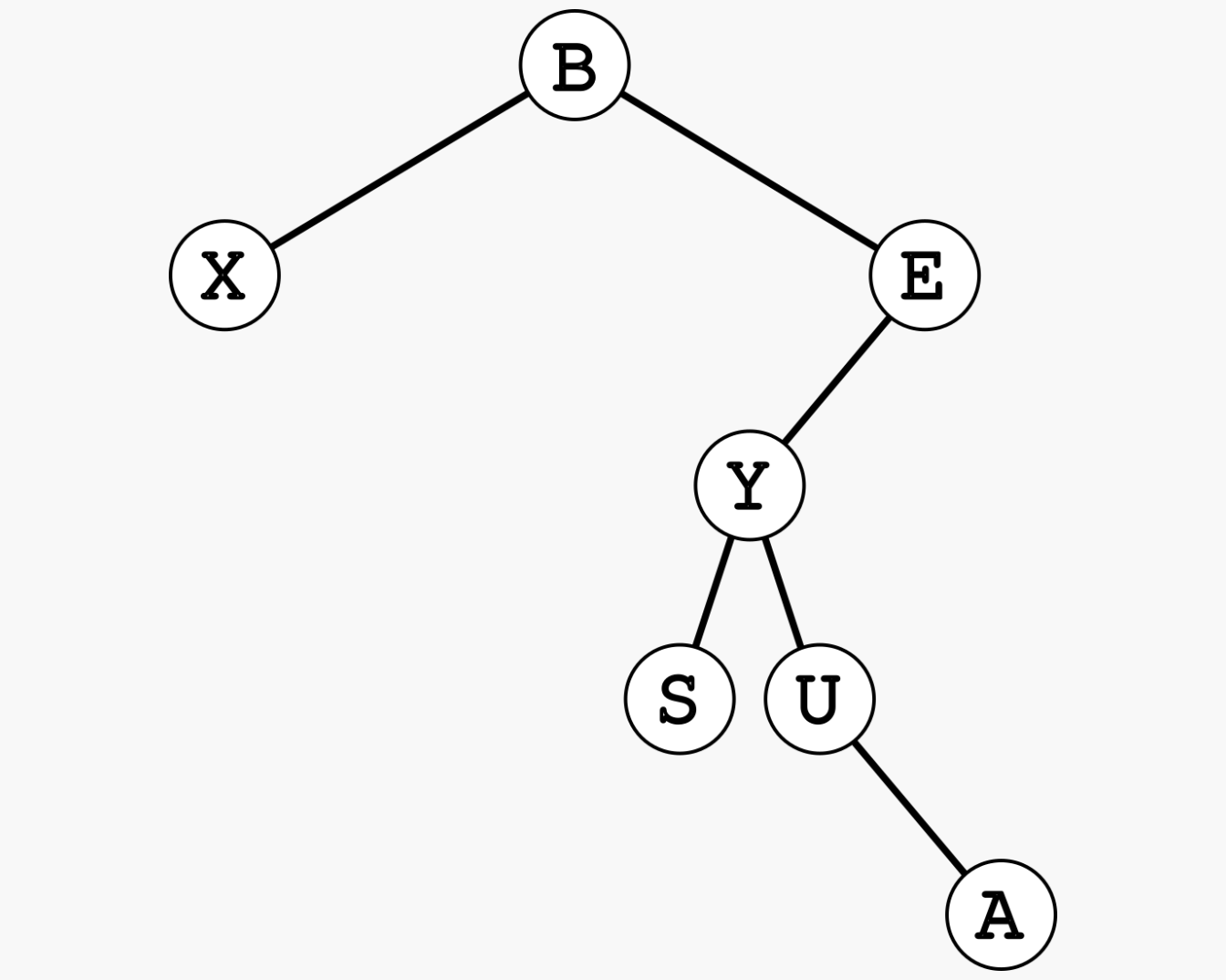
Let x be the time complexity of rebalance. If the time complexity to rebalance is less than the time complexity of search, $O(\log_2 n)$, in the binary search tree then:

$O(\log_2 n) + x = O(\log_2 n)$ where $x \leq O(\log_2 n)$,

And the overall efficiency of insert or search based operations is no more than $O(\log_2 n)$.

If a binary search tree is not properly balanced, the efficiency of search can degenerate beyond the expected worst-case efficiency. The unbalanced tree degenerates towards becoming a linked-list rather than remaining a binary tree, so the efficiency of search in a *degenerate BST* tends towards $O(n)$ in the worst case; however, a degenerate BST is only possible if rebalancing is not properly or regularly performed.

The balanced property and the balancing algorithm are therefore key in maintaining optimal search efficiency in a binary search tree and the efficiency of the rebalancing algorithm will dominate the efficiency of any insertion operation in the tree.



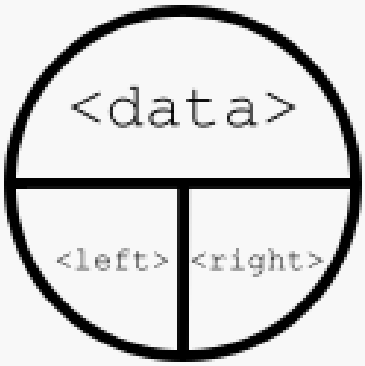
A degenerate tree

Binary Tree Implementation

Binary trees share structural elements with a linked list. Both the linked list and the tree require an anchor, *i.e.* head versus root, to maintain access to the data structure in memory, use references to link together elements within each data structure, and use similar node data structures to maintain data and relationships in the data structure.

Binary Tree Node

The binary tree node has the same structural elements as a doubly linked-list node, *i.e.* a data element and two pointers to nodes of the same class, but the concept of the structure is slightly different as signified by the unique labels used for the binary tree node.



Binary tree node structure

```
class BinaryTreeNode {
    Object data;            // the data in this node
    BinaryTreeNode left;    // node to left if it exists
    BinaryTreeNode right;   // node to right if it exists
}
```

In a tree class we need minimally to track the root node and provide internal operations to maintain the integrity of the tree itself such as a balancing operation. For more sophisticated trees, more methods and more sophisticated node properties may be needed.

```
class BinaryTree {
    BinaryTreeNode root;                    // root of the tree

    void balance( BinaryTreeNode lroot );   // balance algorithm
    ...
}
```

Traversal

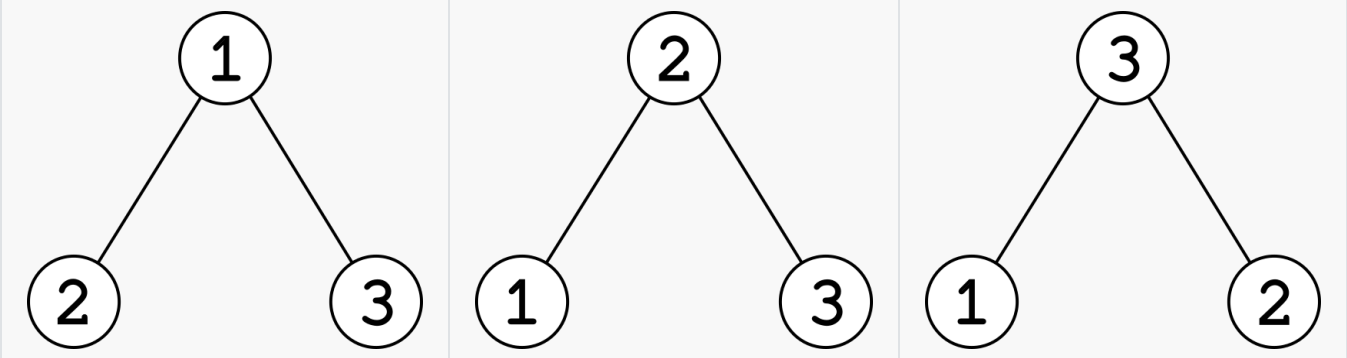
When dealing with a linked list, we needed a methodology to traverse the list; however, for a linked-list, any traversal methodology is very straightforward and simply requires an iterative approach. For a tree, such an approach is not possible because for each node there are multiple paths that can be traversed and as a result we need to be considerate of the varied (breadth-first, depth-first, boundary, and diagonal) general traversal strategies that may be used to traverse a binary tree and each may be further structured by a specific order.

It is important to consider that **inspecting** a node is a separate operation from visiting node. We can visit a node and then visit one or more of its children before actually making any comparison of a value stored in the node itself. Given this consideration, the order we visit nodes and the point at which in the visiting process we inspect a node becomes very important.

Depth-First Search (DFS)

A depth-first approach emphasizes descending the tree first before branching out and examining alternatives. However, this comes at the cost of potentially exploring an extremely deep tree before considering a more desirable decision adjacent to any initial choice.

These depth-first traversals are known as **pre-order traversal**, **in-order traversal**, and **post-order traversal**. The prefix in these conventions describes when the local root is inspected with respect to the children, so pre-order means the root is inspected before its child nodes are inspected, in-order means the root is inspected between inspection of the two children, and post-order means the root is inspected after both children are inspected.



Pre-order traversal	In-order traversal	Post-order traversal

In the traversal diagrams above, the node labeled 1 is visited first, the node visited 2 is visited second, and the node labeled 3 is visited third; and this process is performed recursively for each node. By default, these traversals follow a left-to-right convention but a "reverse" convention of right-to-left, *i.e.* reverse pre-order, reverse in-order, and reverse post-order, can be specified and implemented. These multiple traversal approaches reflect the increased complexity of a tree over a linked-list; however, it leaves us to consider whether this complexity is a benefit or a burden.

> **Inspection**
>
> In the following traversal discussion, we will use the term "inspect" or "inspection" to indicate when an action is performed on the local root. If we were reading data from the root or deleting the root, these actions would be performed at the time of inspection. We make this distinction because the action must be performed at a specific phase during the traversal if the order is to be preserved.

Pre-Order

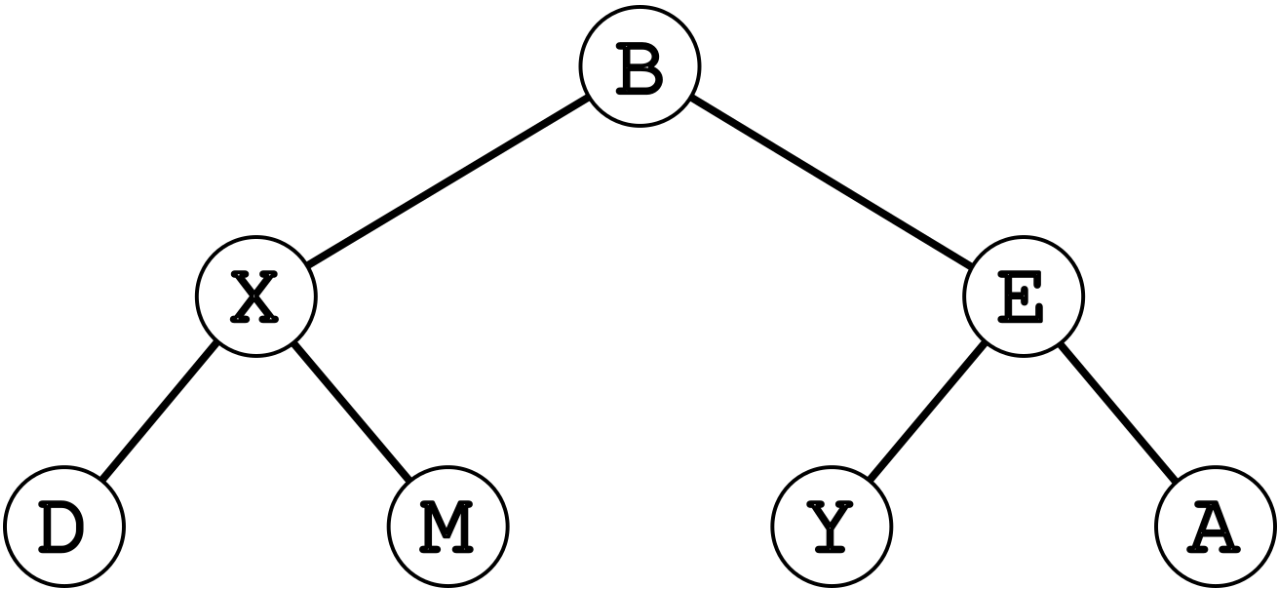
Pre-order traversal means that the local root is inspected before any of its children are visited.

If we transcribe tree nodes into an array using pre-order traversal, the order of nodes in the array will reflect the construction order of the tree. In other words, we can flatten a tree into an array and then reconstruct the tree from the flattened array. This is useful because we often need to serialize, *i.e.* convert data into a sequence of symbols, for transmission, and by using pre-order traversal, we could serialize a tree, broadcast the tree as an array to another system, and reconstruct the tree on the other end of transimission.

The recursive pre-order traversal algorithm is as follows:

```
Preorder(root) :  root -> left -> right
    Inspect(root)
    Preorder(root.left)
    Preorder(root.right)
```

The following example illustrates pre-order traversal over the entire tree:



	Step	Actions
1	Inspect B	Descend left to X
2	Inspect X	Descend left to D
3	Inspect D	Ascend to X. Descend right to M.
4	Inspect M	Ascend to B. Descend right to E.
5	Inspect E	Descend left to Y
6	Inspect Y	Ascend to E. Descend right to A.
7	Inspect A	

In-Order

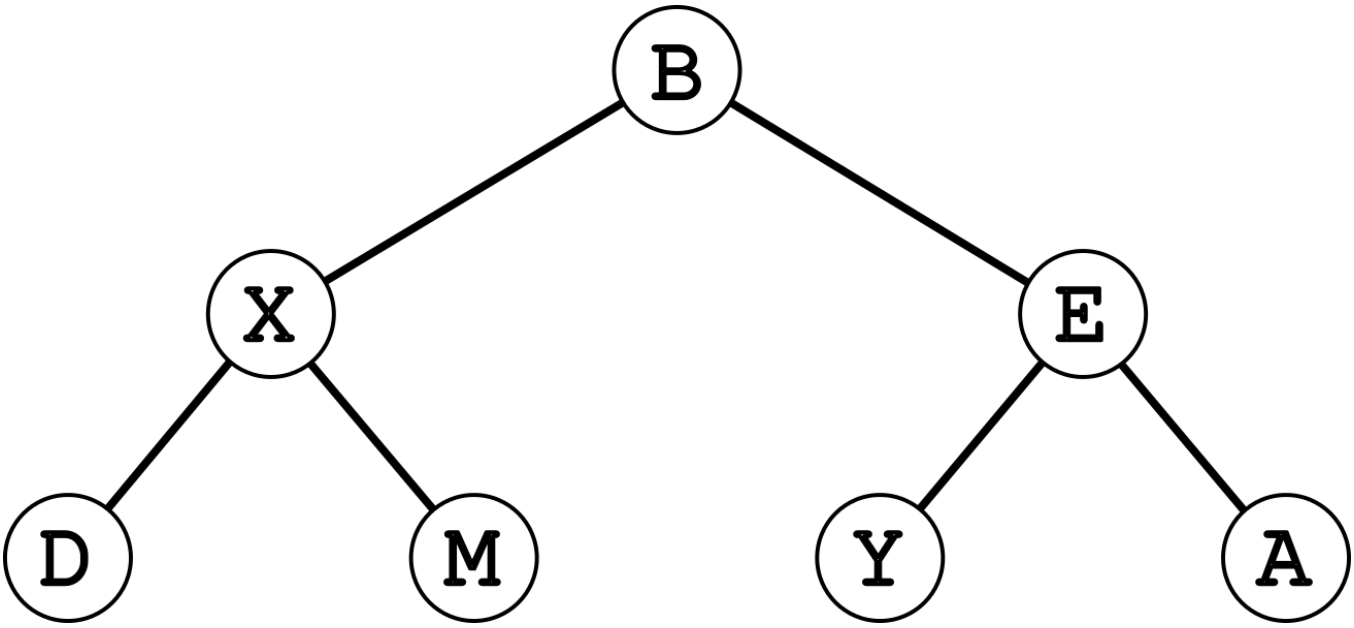
In-order traversal means that the local root is inspected in between visiting its children.

If we transcribe an ordered tree's nodes into an array using in-order traversal, the array itself will be in order. In other words, in-order traversal is capable of flattening an ordered tree into an ordered array.

The recursive in-order traversal algorithm is as follows:

```
Inorder(root) :  left -> root -> right
    Inorder(root.left)
    Inspect(root)
    Inorder(root.right)
```

The following example illustrates in-order traversal over the entire tree:



	Step	Actions
		Descend left to X. Descend left to D.
1	Inspect D	Ascend to X.
2	Inspect X	Descend to M.
3	Inspect M	Ascend to B.
4	Inspect B	Descend right to E. Descend left to Y.
5	Inspect Y	Ascend to E
6	Inspect E	Descend right to A.
7	Inspect A	

Post-Order

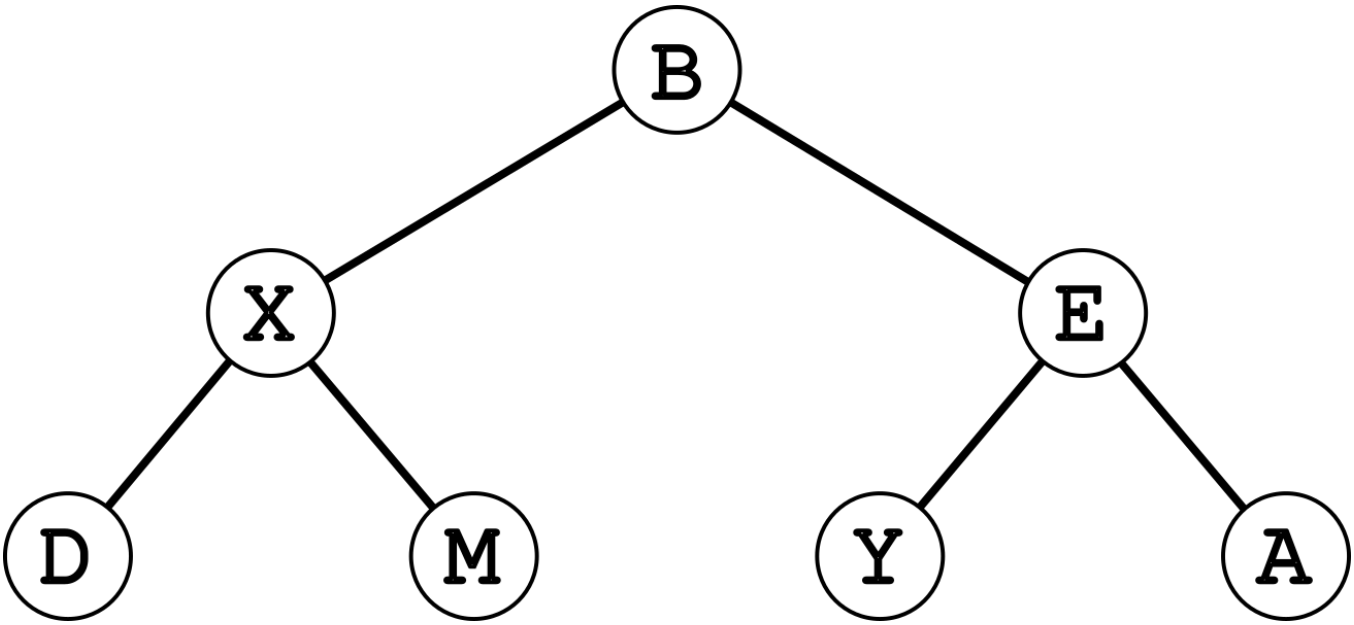
Post-order traversal means that the local root is inspected after all of its children are visited.

When we need to delete the contents of the tree, we need to start **pruning** at the leaves and work back up to the root of the tree in order to ensure full deletion of the tree. We also may use the same approach for pruning specific branches of the tree. To support this, we would need to inspect the children before inspecting their local root, and this is exactly what post-order traversal guarantees.

The recursive post-order traversal algorithm is as follows:

```
Postorder(root) :  left -> right -> root
    Postorder(root.left)
    Postorder(root.right)
    Inspect(root)
```

The following example illustrates post-order traversal over the entire tree:



	Step	Actions
		Descend left to X and D
1	Inspect D	Ascend to X. Descend right to M
2	Inspect M	Ascend to X
3	Inspect X	Ascend to B. Descend right to E. Descend left to Y
4	Inspect Y	Ascend to E. Descend to right A
5	Inspect A	Ascend to E.
6	Inspect E	Ascend to B.
7	Inspect B	

Breadth-First Search (BFS)

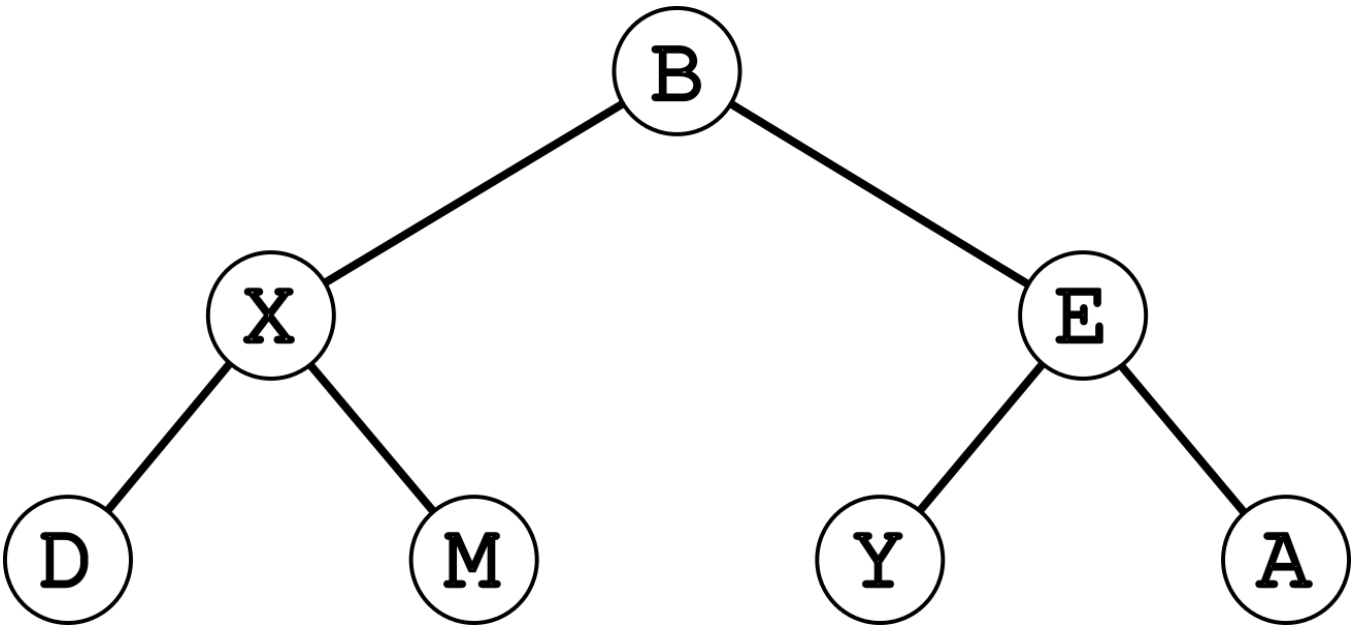
Rather than prioritizing iteration down the tree, **breadth-first search (BFS)** traversal prioritizes traversal at the current level, known also as **level-order traversal**, and it ensures that all elements at the current level are inspected before any descent down the tree is made. There are no direct relationships among elements at the same level which requires some other data structure to support BFS traversal. In general, a queue can aid in BFS where when a node is inspected, its children are enqueued for future inspection and inspection occurs based on the queueing order.

There is no simple recursive algorithm for level-order traversal as there are no links between elements in the same level; however, the algorithm is still quite simple.

```
Levelorder(root) :
    queue ← queue.empty()
    queue.enqueue(root)
    while not queue.isEmpty()
        node ← queue.dequeue()
        Inspect(node)
        if node.left ≠ null
            queue.enqueue(node.left)
        if node.right ≠ null
            queue.enqueue(node.right)
```

Adapted from https://en.wikipedia.org/w/index.php?title=Tree_traversal#Breadth-first_search_2

The following example illustrates level-order traversal over the entire tree:



	Step	Actions
1	Inspect B	Enqueue X, Enqueue E

	Step	Actions
2	Dequeue X, Inspect X	Enqueue D, Enqueue M
3	Dequeue E, Inspect E	Enqueue Y, Enqueue A
4	Dequeue D, Inspect D	
5	Dequeue M, Inspect M	
6	Dequeue Y, Inspect Y	
7	Dequeue A, Inspect A	

BFS is used to traverse decision trees used in very simple heuristic based AIs.

For example, a chess engine would not use a DFS approach because trees do not model cycles, so there are an infinite number of board positions. As as result, DFS might be slow and would requires that we explore the entire tree for the "best" decision. However. some decision trees are infinitely deep, *e.g.* Chess and Go, so DFS will never stop.

If planning time is bounded, often BFS will give a better outcome than DFS because it can sample a more varied number of distinct choices, and if a scoring metric is available, it can choose the highest scoring option. BFS can look several layers deep in a relatively short time to find the best choice among available choices rather than exploring one option to its conclusion. For really deep decision trees, a level order traversal will better support decision making over DFS.

Binary Search Tree Implementations

There are multiple approaches to maintaining a balanced BST, but two established approaches are AVL and Red-Black Trees.

Adelson-Velsky and Landis (AVL) Tree (1962)

The Adelson-Velsky and Landis (AVL) approach defines a **balance factor**, *i.e* a binary tree is balanced if the absolute difference of heights of left and right subtrees at any node is no greater than 1. In other words, using a recursive descent through the tree, each and every left and right subtree must remain balanced for every subroot in the tree and this is measured by taking the difference between the heights of the left and right subtrees.

Where n as the number of nodes in the tree, recomputing the balance factor would be a relatively expensive $O(n)$ operation if it is necessary to recompute the balance factor for every node in the tree. However, it is not necessary to recompute the balance factor for every node if two observations are made. First, the balance factor can be maintained as a property of each node and so we have access to the previously computed balance factor whenever we need to rebalance. Second, the balance factor only needs to be recomputed in the subtrees along the path from root to the newly added (or removed) node as all other paths through the tree remain unchanged and will have the same balance factor as what is stored in their nodes.

The key observation to make rebalancing efficient for the AVL tree is that we do not need to rebalance the entire tree and instead, in the worst case, can rebalance along a single path through the tree to produce a new tree of minimum height. If we only have to rebalance along a single path and the original tree was of minimum height, then the efficiency of rebalance can be as fast as $O(\log_2 n)$ where n is the number of nodes in the tree because the height of the new binary tree is no greater than $\log_2 n + 1$. So how can we optimally rebalance local subtrees at $O(\log_2 n)$ without rebalancing the entire tree at $O(n)$ and the answer is to use rotations.

Rotations

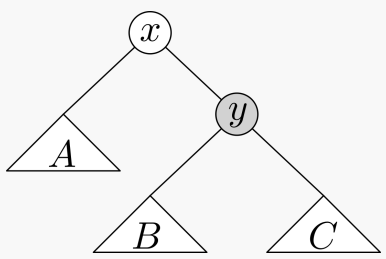
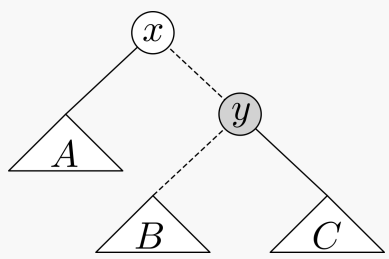
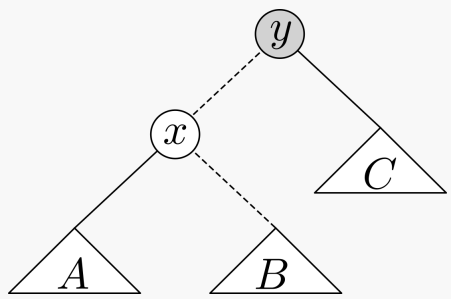
A rotation is like grabbing a node adjacent to the subtree's root, promoting that adjacent node to be the new subtree's root, and then rehanging the entire subtree from that node. Since we are working with references, we can simply reassign left and right elements of a node in constant time, so if we avoid traversing nodes and only need to perform a rotation to rebalance, then rebalancing a specific subtree can be carried out in constant time and if we reached the worst case of needing to rebalance the entire tree, we would need only to rotate at most $\log_2 n$ subtrees.

The following discussion is a summarization of [Decker 1989]

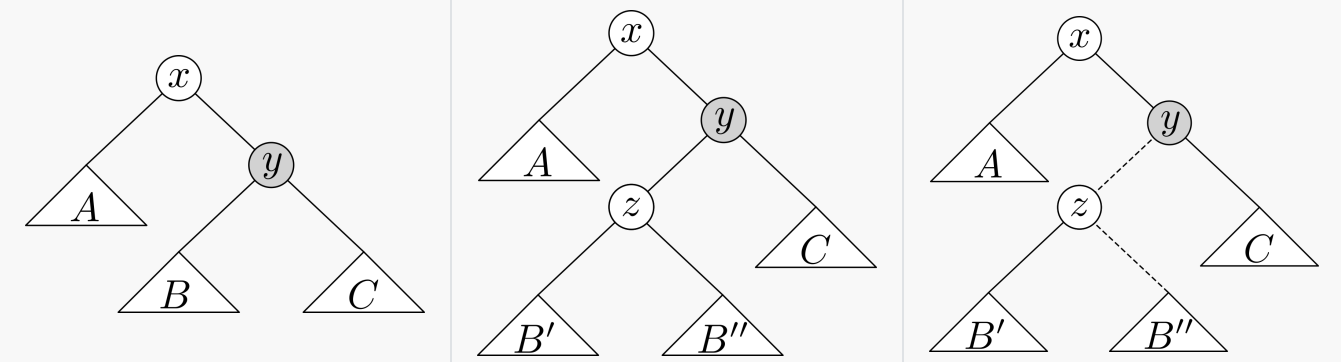
There are two cases where rotations are necessary and each has two flavors; each is a mirror image of the other.

We will assume that the imbalance was caused by the right subtree of a node x being too high, either by an insertion to the right of x or a deletion to the left of x. We denote the root of the right subtree of x by y. The other case, where the imbalance was caused by the left subtree of node x being too high, is dealt with by interchanging left and right in the subsequent discussion.

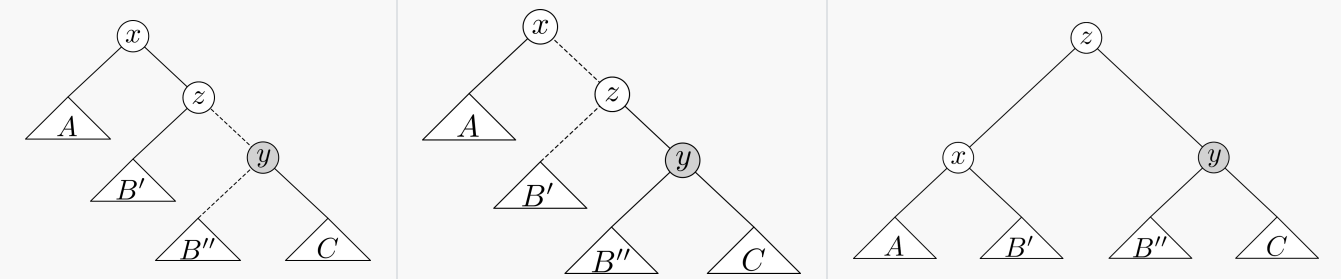
Case 1 : Suppose the right subtree of y has height larger than that of the left subtree of y caused by the deletion of a node from subtree A or insertion into subtree C. A left rotation preserves the left-right order A, x, B, y, C so the BST is preserved by left rotations, and the left rotation in this case restores the balance at node x. Restoring the balance at x may change the height of the subtree rooted at x, so we may have to continue the balancing process at the parent of x, and so on up to the root of the AVL tree.

		
Rotating left by promoting y to be the root of the subtree originally defined by x as the root. All elements of B are greater than or equal to x and less than y	To rotate, we can reassign the references associated with the right child of x and the left child of y. Since y must be greater than or equal to x, x also qualifies to be the left child of y	By reassigning the left child of y to be x and simultaneously reassigning the right child of x to be B (formerly the left child of y), the subtree is rebalanced now with y as the root of the subtree.

Case 2 : Suppose that the imbalance at x is caused by the left subtree of y being too high. A simple rotation in this case is not sufficient to restore balance at x. Instead, we need to perform two rotations, beginning further down the tree. We expand subtree B into its root and subtrees B' and B'' to illustrate the first rotation on the right child of x where the expanded subtree with root z is rotated with its parent y. A second rotation is then performed at z such that x and the left child of z are now balanced to the left of z, y and the right child of y are now balanced to the right of z, and z is now the root of the overall subtree.



Suppose the left subtree of y, B, is too high	We expand subtree B into its root z and subtrees B' and B''	We break the edge (y, z) and the edge (z, B'')



We can then hang y from the right of z and B'' from the left of y	We can then break the edge (x, z) and the edge (z, B') to perform a left rotation on z	With the left rotation on z the overall subtree is rebalanced with z newly promoted to the root of the subtree. Recall that z began as the root node of the subtree B.

Red-Black Tree (1978)

The Red-Black Tree is an extension of the AVL tree concept and was invented by Guibas and Sedgewick where the primary benefit of Red-Black trees is their additional complexity helps reduce the frequency of rebalancing. The Red-Black tree uses an additional property, "color", to determine if balancing can stop before reaching the root. In the Red-Black tree, every node has a color and color generally alternates at each level. Further balancing is necessary if when ascending the tree during the balancing operation, two successive nodes have the same color.

Here is a high level summary of the Red-Black tree rules:

1. Every node is either red or black (or practically either black and not black).
2. Empty leaf nodes (NIL nodes) are added whenever a new terminal node is added. All empty leaf nodes are always colored black regardless of the color of the newly added terminal node.

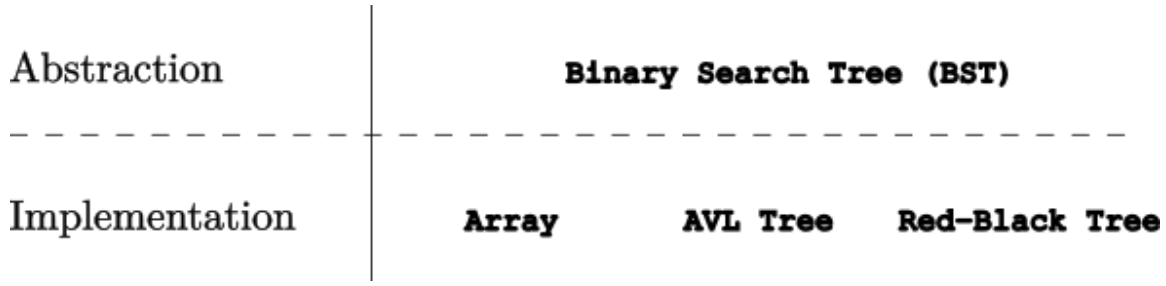
3. A red node can never have a red child
4. Every path from a local root to any descendant NIL node goes through the same number of black nodes.
5. If a node N has exactly one child, the child must be red, because if it were black, its NIL descendants would sit at a different black depth than N's NIL child, violating requirement 4.

Search in an Unordered Tree

Even if a tree is unordered, we may still search it but with a worst case search efficiency approaching a linear approach regardless of its structure. In other words, if a tree is unordered, we have to search every node before determining that the search value does not exist in the tree and our efficiency is no better than search in an unordered array.

Binary Search Tree Abstraction

As suggested previously, trees can be represented using reference based binary tree structure or using an array. A binary search tree must remain balanced to ensure that it maintains its worst-case search efficiency of $O(\log_2 n)$ and there are at least three candidate implementations to ensure a binary search tree is not degenerate.



In other words, a BST can be maintained using an ordered array, an AVL tree or a red-black tree; however, users of the BST do not need to know what data structure is supporting its underlying implementation if a binary search tree abstraction is provided. A typical BST interface might have the following methods:

```
void insert( element ) : Takes an element as input, inserts that element into the binary search
tree in order, returns nothing

boolean contains( element ) : Takes an element as input, searches the binary search tree to
determine whether or not the element exists in the search tree, returns true if the element is
in the tree otherwise returns false.
```

While the efficiency of search in the array implementation is $O(\log_2 n)$ where n is the size of the input, the reason not to use the array becomes clear with the $O(n)$ efficiency of insert. Since the array must remain ordered in order for the array to be a BST, we have to assume that we must shift n elements in the worst-case to make room for the new element to insert regardless of whether there is spare space allocated. This is why there is extensive study of reference based structures that are capable of both insert and search in $O(\log_2 n)$ worst-case time complexity.

Review

1. Define the following terms in terms of the morphology of a tree: Node (inner/outer, terminal/branch,

parent/child), Root, Branch (edge), Leaf

2. What does it mean for two nodes to be adjacent? What does it mean for two nodes to be connected? What is a path through the tree?
3. Define distance in a tree.
4. Define height or depth of a tree. What is a level in a tree?
5. Define the term degree with respect to a node, *i.e.* define "degree of a node". Define the term degree with respect to a tree, *i.e.* define "degree of a tree".
6. What is the difference between an outgoing and an incoming branch? How many incoming branches do nodes in a tree have? What is the maximum number of outgoing branches for a given node in a binary tree and what is the relationship to degree? Associate these arguments with the statement, "a tree is a directed graph".
7. What is the meaning of ancestor and descendant and parent and child in a tree in terms of levels and connectedness? What are the relationships between parent and child nodes specifically with respect to adjacency and what are the relationships between ancestor and descendant specifically with respect to paths?
8. What is the significance of an ancestor's position in the tree with respect to the root and any descendants.
9. What can generally be said about the ancestral relationship between the root and all nodes in the tree. Justify your answer.
10. Define what it means to be balanced in terms of all the tree's subtrees with respect to the heights of those subtrees' left and right subtrees. What is a rotation in a binary tree balancing algorithm?
11. How many nodes n are in a complete binary tree with depth d? Justify your answer.
12. What is the mathematical relationship between and the maximum number of nodes in a level ℓ for a binary tree? What is the mathematical relationship between the length of the path from root to any leaf in a complete binary tree with depth d?
13. A degenerate tree is exceedingly unbalanced. Explain what data structure the degenerate tree becomes like and why. Why would a tree become a degenerate tree?
14. Justify why a balanced tree performs insert and search operations in logarithmic time complexity with a worst case analysis based on the length of a path in the tree.
15. What is the relationship between the number of nodes in a balanced tree and its depth?
16. Why do we want to ensure that a binary tree remains balanced throughout its usage?
17. Insert sample data into an ordered binary tree and show the resulting tree.
18. Show tree traversal using preorder, postorder, and inorder traversal by recording the order in which nodes are visited given the traversal strategy.
19. Show tree traversal using level order traversal by recording the order in which nodes are visited
20. Which form of search is preorder traversal: depth first (DFS) or breadth first (BFS)? Justify your answer. It is depth first because...
21. Which form of search is inorder traversal: depth first (DFS) or breadth first (BFS)? Justify your answer. It is depth first because...

22. Which form of search is postorder traversal: depth first (DFS) or breadth first (BFS)? Justify your answer. It is depth first because...
23. Which form of search is level order traversal: depth first (DFS) or breadth first (BFS)? Justify your answer. It is breadth first because...
24. What is meant by depth first search (DFS) as opposed to breadth first search (BFS)? How do these two different search strategies emerge from the choice of traversal you implement?
25. What characteristics of a binary tree are required for a binary tree to be considered a binary search tree?
26. Why can we characterize the search efficiency of a BST as the length of the longest path through the tree? Why is the search efficiency not the length of all paths through the tree?
27. Given a balanced BST, what is the length of the longest path given n where n is the number of elements in the BST. Justify your answer.
28. Why is balance critical to maintaining a BST's search efficiency? How does whether or not a BST is balanced affect the length of the longest path through the BST?